BOAZ-STUDY

Document Évaluateur – Usage Interne

CAHIER DE RECETTE

Test Technique – Ingénieur(e) DevOps / SRE

Projet InfraFlow – Déploiement et Opération d'une Infrastructure Microservices

Version	1.0
Total points	130 points – 30 scénarios
Usage	Évaluateur uniquement – confidentiel

1. Introduction & Barème

1.1 Répartition des points

Catégorie	Max	Obtenu	%	Statut
Docker – Dockerfiles + docker-compose	20			
CI/CD – GitHub Actions pipeline	25			
Kubernetes – manifestes + déploiement	30			
Monitoring – Prometheus + Grafana	20			
Script Bash deploy.sh	15			
Sécurité DevSecOps	15			
Documentation README	5			
TOTAL	**130**			

1.2 Seuils de décision

Décision	Score	Signification
⏳ CONFIRMATION	≥ 85% (≥ 111 pts)	Maîtrise complète – Candidat excellent
☑ ACCEPTABLE	70–84% (91–110)	Compétent, quelques lacunes mineures
⚠ À REVOIR	50–69% (65–90)	Lacunes significatives

Décision	Score	Signification
❌ ÉLIMINATOIRE	< 50% (< 65 pts)	Niveau insuffisant ou critère éliminatoire raté

1.3 Critères éliminatoires

L'échec à l'un des tests marqués ⚠ **ÉLIMINATOIRE** peut entraîner une élimination immédiate :

- Docker Compose non fonctionnel → infrastructure de base absente
- CI/CD GitHub Actions absent → pas de pipeline
- Kubernetes : aucun pod Running → déploiement échoué
- Secrets en dur dans les fichiers → pratique inacceptable en production

2. Docker & Docker Compose (20 pts)

DOCK-001 – Dockerfiles optimisés pour chaque service ⚠ ÉLIMINATOIRE	
Catégorie	Docker – Dockerfile
Prérequis	Code source accessible
Étapes	1. Ouvrir api-service/Dockerfile et web-service/Dockerfile 2. Vérifier l'utilisation d'une image de base officielle et légère (alpine de préférence) 3. Vérifier qu'aucun processus ne tourne en tant que root (USER directive) 4. Vérifier la présence d'un .dockerignore 5. Lancer : docker build -t api-service ./api-service && vérifier le build réussi
Résultat attendu	**Images buildées sans erreur – processus non-root – .dockerignore présent**
Points / Résultat	**8 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

DOCK-002 – Infrastructure complète démarrable en une commande ⚠ ÉLIMINATOIRE	
Catégorie	Docker Compose – démarrage complet
Prérequis	docker et docker compose installés
Étapes	6. Exécuter : docker compose up -d 7. Vérifier : docker compose ps – tous les services en état healthy ou running 8. Tester la réponse : curl http://localhost:8080/get (ou l'URL configurée) 9. Vérifier la présence d'un fichier .env.example (jamais de secrets en dur dans docker-compose.yml)
Résultat attendu	**Tous les services UP en une commande – réponse HTTP 200 – pas de secrets en clair**
Points / Résultat	**8 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

DOCK-003 – Health checks et réseau Docker configurés	
Catégorie	Health checks + réseau

Prérequis	docker-compose.yml accessible
Étapes	10. Vérifier la présence de healthcheck sur chaque service dans docker-compose.yml 11. Vérifier qu'un réseau Docker personnalisé est défini (networks:) 12. Vérifier depends_on avec condition: service_healthy
Résultat attendu	**Health checks présents, réseau personnalisé défini, dépendances correctement configurées**
Points / Résultat	**4 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

3. CI/CD – GitHub Actions (25 pts)

CICD-001 – Repository GitHub accessible ⚠ ÉLIMINATOIRE	
Catégorie	Workflow file présent et structuré
Prérequis	Accès au repository GitHub
Étapes	13. Vérifier la présence de .github/workflows/ci-cd.yml 14. Vérifier les triggers : on push (main) et on pull_request 15. Vérifier la présence des 4 jobs : lint-and-test, build, security-scan, push
Résultat attendu	**Fichier workflow présent avec les 4 jobs définis et les bons triggers**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

CICD-002 – Linting automatique du code infra ⚠ ÉLIMINATOIRE	
Catégorie	Job lint-and-test
Prérequis	Workflow visible dans GitHub Actions
Étapes	16. Vérifier que le job lint-and-test utilise hadolint pour les Dockerfiles 17. Vérifier l'utilisation de shellcheck sur les scripts Bash 18. Vérifier le résultat de ce job dans GitHub Actions (doit être vert sur main)
Résultat attendu	**Job vert – hadolint et shellcheck passent sans erreur critique**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

CICD-003 – Images buildées et taguées avec le SHA commit ⚠ ÉLIMINATOIRE	
Catégorie	Job build – images taguées avec SHA
Prérequis	Workflow exécuté au moins une fois
Étapes	19. Vérifier dans le job build que le tag utilise github.sha 20. Vérifier que les images sont buildées en utilisant le cache Docker si possible

Résultat attendu	**Tag contient le SHA du commit – ex: image:sha-abc1234**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

CICD-004 – Scan de vulnérabilités avec Trivy ⚠ ÉLIMINATOIRE	
Catégorie	Job security-scan – Trivy
Prérequis	Workflow exécuté au moins une fois
Étapes	21. Vérifier la présence du step Trivy (action aquasecurity/trivy-action ou commande trivy) 22. Vérifier que le pipeline échoue sur les vulnérabilités CRITICAL (exit-code: 1) 23. Vérifier la présence du rapport de scan dans les artefacts du job
Résultat attendu	**Trivy présent – pipeline bloque sur CRITICAL – rapport généré**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

CICD-005 – Pas de secrets en clair dans le workflow ⚠ ÉLIMINATOIRE	
Catégorie	Job push – credentials sécurisés
Prérequis	Code source du workflow
Étapes	24. Vérifier que les credentials Docker Hub ou GHCR utilisent ${{ secrets.XXX }} 25. Vérifier que le job push ne s'exécute QUE sur la branche main (condition if: github.ref == 'refs/heads/main')
Résultat attendu	**Credentials via secrets uniquement – push conditionnel à main**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

4. Kubernetes (30 pts)

K8S-001 – minikube ou kind disponible ⚠ ÉLIMINATOIRE	
Catégorie	Cluster local démarré + namespace
Prérequis	minikube ou kind installé sur la machine
Étapes	26. Lancer minikube start ou kind create cluster 27. Exécuter : kubectl apply -f k8s/ 28. Vérifier : kubectl get namespaces – namespace infraflow présent
Résultat attendu	**Cluster démarré – namespace infraflow créé**

Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

K8S-002 – Tous les pods du namespace infraflow en Running ⚠ ÉLIMINATOIRE

Catégorie	Pods en état Running
Prérequis	Cluster démarré, manifestes appliqués
Étapes	29. Exécuter : kubectl get pods -n infraflow 30. Vérifier que tous les pods sont en état Running (pas Pending ou CrashLoopBackOff) 31. Vérifier le nombre de replicas (minimum 2 par service)
Résultat attendu	**Tous les pods Running avec 2+ replicas par service**
Points / Résultat	**8 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

K8S-003 – Liveness/readiness probes et resources définies ⚠ ÉLIMINATOIRE

Catégorie	Probes + Resources
Prérequis	Code source des manifestes
Étapes	32. Ouvrir k8s/api-service/deployment.yml 33. Vérifier la présence de livenessProbe et readinessProbe 34. Vérifier la présence de resources.requests et resources.limits 35. Vérifier que le tag d'image n'est pas :latest
Résultat attendu	**Probes présentes, resources définies, pas de tag :latest**
Points / Résultat	**7 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

K8S-004 – HorizontalPodAutoscaler configuré sur api-service

Catégorie	HPA fonctionnel
Prérequis	Metrics server disponible (minikube addons enable metrics-server)
Étapes	36. Exécuter : kubectl get hpa -n infraflow 37. Vérifier que le HPA cible api-service avec CPU threshold 70%
Résultat attendu	**HPA présent et configuré correctement**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

K8S-005 – web-service accessible depuis l'hôte

Catégorie	Service accessible
Prérequis	Cluster démarré

Étapes	38. Exécuter : kubectl get svc -n infraflow 39. Accéder au web-service via minikube service web-service -n infraflow --url OU kubectl port-forward 40. Vérifier la réponse HTTP 200
Résultat attendu	**web-service accessible depuis l'hôte – réponse HTTP 200**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

5. Monitoring – Prometheus + Grafana (20 pts)

MON-001 – Helm installé, cluster Kubernetes actif ⚠ ÉLIMINATOIRE	
Catégorie	Installation via Helm
Prérequis	helm disponible dans le PATH
Étapes	41. Vérifier la commande helm install dans le README ou un script 42. Exécuter : kubectl get pods -n monitoring 43. Vérifier que les pods prometheus et grafana sont en Running
Résultat attendu	**kube-prometheus-stack déployé via Helm – pods Running dans le namespace monitoring**
Points / Résultat	**8 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

MON-002 – Grafana en Running	
Catégorie	Grafana accessible + dashboard
Prérequis	kubectl port-forward disponible
Étapes	44. Exécuter : kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring 45. Ouvrir http://localhost:3000 dans le navigateur 46. Vérifier la connexion (admin / mot de passe configuré) 47. Vérifier la présence d'un dashboard affichant CPU et mémoire des pods du namespace infraflow
Résultat attendu	**Grafana accessible – dashboard CPU/mémoire des pods infraflow visible**
Points / Résultat	**7 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

MON-003 – Code source des manifestes ou Helm values	
Catégorie	Alerte PrometheusRule
Prérequis	Cluster avec monitoring déployé

Étapes	48. Vérifier la présence d'un fichier PrometheusRule (YAML) ou d'une configuration d'alerte dans les Helm values 49. Vérifier que l'alerte cible un scénario réaliste (ex: pod down, CPU élevé)
Résultat attendu	**Au moins une alerte PrometheusRule définie et visible dans Prometheus Alerts**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

6. Script Bash deploy.sh (15 pts)

BASH-001 – Code source accessible ⚠ ÉLIMINATOIRE	
Catégorie	Script exécutable et structuré
Prérequis	scripts/deploy.sh présent
Étapes	50. Vérifier que le fichier commence par #!/usr/bin/env bash 51. Vérifier la présence de set -euo pipefail en début de script 52. Vérifier la présence d'une fonction de vérification des prérequis (kubectl, helm, docker)
Résultat attendu	**Shebang correct, set -euo pipefail présent, prérequis vérifiés**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

BASH-002 – Cluster disponible	
Catégorie	Script fonctionnel de bout en bout
Prérequis	Cluster démarré, script exécutable
Étapes	53. Rendre le script exécutable : chmod +x scripts/deploy.sh 54. Exécuter : ./scripts/deploy.sh 55. Vérifier que le script applique les manifestes et attend que les pods soient Ready 56. Vérifier l'affichage d'un message de confirmation à la fin
Résultat attendu	**Script s'exécute sans erreur – pods Ready – message de confirmation affiché**
Points / Résultat	**10 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

7. Sécurité DevSecOps (15 pts)

SEC-001 – Code source complet ⚠ ÉLIMINATOIRE

Catégorie	Pas de secrets en dur
Prérequis	Repository accessible
Étapes	57. Rechercher les mots 'password', 'secret', 'token', 'key' dans les fichiers YAML, Dockerfile et scripts 58. Vérifier que les valeurs sensibles utilisent des variables d'environnement ou des références à des secrets Kubernetes 59. Vérifier la présence d'un .env.example (pas de .env commité)
Résultat attendu	**Aucun secret en dur dans le code – .env.example présent – .env absent du repository**
Points / Résultat	**8 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

SEC-002 – Manifestes Kubernetes accessibles	
Catégorie	runAsNonRoot dans Kubernetes
Prérequis	Fichiers deployment.yml accessibles
Étapes	60. Ouvrir les fichiers deployment.yml 61. Vérifier la présence de securityContext avec runAsNonRoot: true
Résultat attendu	**securityContext présent avec runAsNonRoot: true sur au moins les pods api-service**
Points / Résultat	**4 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

SEC-003 – hadolint disponible (ou vérification dans le rapport CI)	
Catégorie	Hadolint sans erreur CRITICAL
Prérequis	Rapport du job lint-and-test dans GitHub Actions
Étapes	62. Consulter le rapport du job lint-and-test dans GitHub Actions 63. Vérifier qu'aucune erreur de niveau CRITICAL ou ERROR n'est présente dans le rapport hadolint
Résultat attendu	**Rapport hadolint sans erreur CRITICAL ou ERROR**
Points / Résultat	**3 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

8. Documentation (5 pts)

DOC-001 – README.md présent	
Catégorie	README clair et complet
Prérequis	README.md à la racine du projet
Étapes	64. Lire le README.md

	65. Vérifier la présence des sections : Prérequis, Démarrage rapide (docker-compose + Kubernetes), Monitoring, Structure du projet 66. Suivre les instructions du README et vérifier que le projet se lance correctement
Résultat attendu	**Projet opérationnel en suivant uniquement le README – sans assistance**
Points / Résultat	**5 pts** ☐ PASS ☐ FAIL ☐ N/A
Observations	

9. Grille de Synthèse

9.1 Récapitulatif par catégorie

Catégorie	Max	Obtenu	%	Statut
Docker – Dockerfiles + docker-compose	20			
CI/CD – GitHub Actions	25			
Kubernetes – manifestes + déploiement	30			
Monitoring – Prometheus + Grafana	20			
Script Bash deploy.sh	15			
Sécurité DevSecOps	15			
Documentation README	5			
TOTAL	**130**			

9.2 Checklist technologies

Élément	Présent ?	Fonctionnel ?
Dockerfile non-root + .dockerignore	☐ OUI ☐ NON	☐ OUI ☐ NON
docker-compose.yml – 1 commande, health checks	☐ OUI ☐ NON	☐ OUI ☐ NON
GitHub Actions – 4 jobs (lint, build, scan, push)	☐ OUI ☐ NON	☐ OUI ☐ NON
Trivy scan dans CI – bloque sur CRITICAL	☐ OUI ☐ NON	☐ OUI ☐ NON
Kubernetes – Deployment, Service, ConfigMap, HPA	☐ OUI ☐ NON	☐ OUI ☐ NON
Pods Running avec probes + resources définis	☐ OUI ☐ NON	☐ OUI ☐ NON
Helm – kube-prometheus-stack déployé	☐ OUI ☐ NON	☐ OUI ☐ NON
Grafana – dashboard CPU/mémoire	☐ OUI ☐ NON	☐ OUI ☐ NON
PrometheusRule – alerte configurée	☐ OUI ☐ NON	☐ OUI ☐ NON
Bash deploy.sh – set -euo pipefail + prérequis	☐ OUI ☐ NON	☐ OUI ☐ NON
Pas de secrets en dur (code + CI)	☐ OUI ☐ NON	☐ OUI ☐ NON
runAsNonRoot dans securityContext	☐ OUI ☐ NON	☐ OUI ☐ NON

Élément	Présent ?	Fonctionnel ?
README – projet lancé en < 5 min	☐ OUI ☐ NON	☐ OUI ☐ NON

10. Décision Finale

Score total	_____ / 130 points
Pourcentage	_____ %
Critères éliminatoires	☐ Tous réussis ☐ Échec sur : _____
Décision	☐ ⏳ CONFIRMATION ☐ ☑ ACCEPTABLE ☐ ⚠ À REVOIR ☐ ❌ ÉLIMINATOIRE

Points forts identifiés

Axes d'amélioration

Commentaires de l'évaluateur

Évaluateur : _____	Date : _____